HEALTH LENS: AI MEDICAL REPORT ANALYZER AND EVALUATION FRAMEWORK

A project report submitted in partial fulfilment of the requirements
for the research internship

MASTER OF COMPUTER APPLICATIONS

Submitted by:

ANJALI RANI 24204031205

MANIT Bhopal | aranjalijangra@gmail.com

Submitted to:

Dr. Vinay Raj

Assistant Professor, Department of Computer Applications

DEPARTMENT OF COMPUTER APPLICATIONS
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPPALLI - 620015

SUMMER INTERNSHIP: MAY - JULY 2026

BONAFIDE CERTIFICATE

This is to certify that the project entitled "Health Lens: AI Medical Report Analyzer and Evaluation Framework" is a project work successfully carried out by Anjali Rani (Roll No. 24 204 031 205), student of MANIT Bhopal, during her distance research internship at the Department of Computer Applications, National Institute of Technology, Tiruchirappalli, under the guidance of Dr. Vinay Raj. The internship was offered for a two-month duration starting from 15 May 2026, as per the formal internship offer dated 30 April 2026.

The work reported here includes the analysis, implementation, and evaluation of an AI-assisted medical-report understanding system with retrieval-augmented educational support and a five-metric evaluation framework.

Dr. Vinay Raj	Head of the Department
Mentor	Department of Computer Applications

ABSTRACT

Health Lens is an AI-assisted medical-report analysis and educational question-answering prototype. The system allows an authenticated user to upload a medical report, extract readable text from the document, convert the extracted information into structured report fields, store the analysis in a patient report history, and ask contextual questions through a unified assistant.

The implementation combines a React/Vite frontend, an Express and MongoDB backend, Gemini-based structured generation, PDF text extraction, OCR support, and a retrieval-augmented generation layer over a local medical knowledge base. The assistant separates report interpretation from general educational information, preserves patient-specific values from the uploaded report, and avoids unsafe diagnosis or prescription behaviour.

A recent evaluation module was added to make the project measurable. It decomposes quality into five metrics: Report Extraction Accuracy, Retrieval Hit Rate@5, Answer Correctness, Hallucination Rate, and Safety Compliance Rate. The current prototype baseline reports 28/28 extraction field matches, 3/5 retrieval hits at K=5, and 4/4 safety cases passing, while answer correctness and hallucination remain intentionally human-reviewed. This report documents the system architecture, implementation, frontend workflow, evaluation methodology, limitations, and future research direction.

ACKNOWLEDGEMENT

I express my sincere gratitude to Dr. Vinay Raj, Assistant Professor, Department of Computer Applications, National Institute of Technology, Tiruchirappalli, for his guidance and support throughout this research internship. His direction helped shape the project into an end-to-end system with clear implementation and evaluation goals.

I also thank the Department of Computer Applications, NIT Tiruchirappalli, for providing the academic setting for this internship. I am grateful to MANIT Bhopal for supporting my academic development, and to everyone who provided feedback during the project.

Anjali Rani

(24 204 031 205)

Table of contents

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CBC	Complete Blood Count
JWT	JSON Web Token
LLM	Large Language Model
OCR	Optical Character Recognition
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
UI	User Interface
Zod	TypeScript/JavaScript schema validation library

CHAPTER 1 - INTRODUCTION

Health Lens was developed as a research internship project to explore how an AI system can help users understand medical reports in a safer and more structured way. The system is not intended to diagnose or prescribe treatment. Its purpose is to extract report information, explain medical concepts in patient-friendly language, and provide a measurable foundation for further research.

1.1 Background

Medical reports are often dense, abbreviated, and difficult for non-specialist users to interpret. Patients may see laboratory values, reference intervals, flags such as High or Low, and clinical terms without knowing which values are report-specific and which explanations are general educational background.

A modern LLM can summarize and explain text, but medical use requires guardrails. Patient-specific claims must come from the uploaded report, educational claims should be grounded in reliable source material, and unsafe requests such as diagnosis or prescription must be refused. Health Lens addresses this by combining structured extraction, RAG, and deterministic safety checks.

1.2 Problem Statement

The problem is to design and evaluate a medical-report assistant that can process uploaded reports, store extracted information, answer questions using both report context and an indexed knowledge base, and expose measurable quality indicators for research review.

- Extract key medical parameters, values, reference ranges and status labels from uploaded PDF reports.
- Provide a report-history interface so users can switch assistant context between reports.
- Retrieve educational evidence from a medical PDF knowledge base before generating an answer.
- Separate report interpretation from educational explanation in the assistant response.
- Evaluate extraction, retrieval, answer quality, hallucination and safety using labelled datasets.

1.3 Objectives

- Build a usable frontend for authentication, upload, report history, search, knowledge-base status and chat.
- Implement backend routes for authentication, report upload, knowledge-base synchronization and unified chat.
- Use Gemini structured output with Zod validation for report analysis and chat response classification.
- Implement a RAG layer that supports local vector retrieval and Pinecone-backed retrieval configuration.
- Add a five-metric evaluation harness and document current prototype baselines with sample-size caveats.

1.4 Scope and Contributions

The project scope is an educational prototype for medical-report understanding. It demonstrates a full-stack AI workflow and a research-style evaluation framework, but it does not claim clinical validation or readiness for unsupervised medical deployment.

Contribution	Evidence in project
Full-stack prototype	React dashboard, Express API, MongoDB models, authenticated routes and report history.

Structured analysis	Gemini model output is constrained by a Zod schema for report fields and status labels.
RAG support	Knowledge PDFs are chunked, embedded, cached or indexed, retrieved and cited.
Safety layer	Unified prompts and deterministic urgent-symptom warning logic restrict unsafe outputs.
Evaluation framework	Five metrics, JSON datasets, unit tests and live evaluation commands are implemented.

CHAPTER 2 - TECHNICAL BACKGROUND

This chapter summarizes the technical concepts that shaped the system: structured extraction, retrieval-augmented generation, schema validation, embeddings, and medical safety boundaries.

2.1 Structured Medical Report Extraction

Structured extraction converts raw report text into predictable fields. In Health Lens, the target structure includes report type, summary, abnormal values, suggestions, suggested questions and a parameter array. Each parameter stores the test name, observed value, reference range and status.

2.2 Retrieval-Augmented Generation

RAG reduces unsupported educational answers by retrieving relevant chunks before generation. Instead of asking the LLM to rely only on its pretraining, the backend searches the indexed medical knowledge base and passes the retrieved evidence to the assistant prompt.

2.3 Embeddings and Similarity

The project represents chunks and questions as dense embedding vectors. Similarity search ranks chunks by vector closeness. The local provider uses cosine similarity over cached vectors; the Pinecone path supports vector database search when configured.

2.4 Medical Safety Boundary

The assistant is designed for educational support, not clinical decision-making. It refuses diagnosis and prescription requests, separates patient-specific report interpretation from general information, recommends professional review, and adds an emergency warning for urgent symptom patterns.

Concept	Role in Health Lens
Zod schema	Constrains LLM output into expected report and chat structures.
Temperature 0	Improves repeatability during evaluation and structured generation.
JWT authentication	Protects report upload, history and chat endpoints.
Top-K retrieval	Selects the most relevant chunks for prompt context and citation.
Manual review	Handles answer correctness and hallucination labels where exact matching is inadequate.

CHAPTER 3 - SYSTEM REQUIREMENTS AND ARCHITECTURE

3.1 Functional Requirements

- Allow users to register, log in and access report features through token-protected API calls.
- Upload a medical PDF and process it into extracted text and structured analysis fields.
- Maintain a searchable history of uploaded reports and report-specific chat history.
- Index medical knowledge-base PDFs and report readiness status to the frontend.
- Answer user questions with optional report context and retrieved educational evidence.
- Provide evaluation commands for metric calculation and manual answer review.

3.2 Non-Functional Requirements

Requirement	Implementation response
Reproducibility	Metric formulas are pure functions with deterministic unit tests.
Safety	Prompts, structured medical context labels and deterministic emergency warnings are used.
Modularity	Routes, controllers, services, models and evaluation modules are separated.
Extensibility	RAG provider can be local or Pinecone; datasets can grow by adding JSON cases.
Usability	Dashboard shows report counts, findings, status, selected report and assistant context.

3.3 Overall Architecture

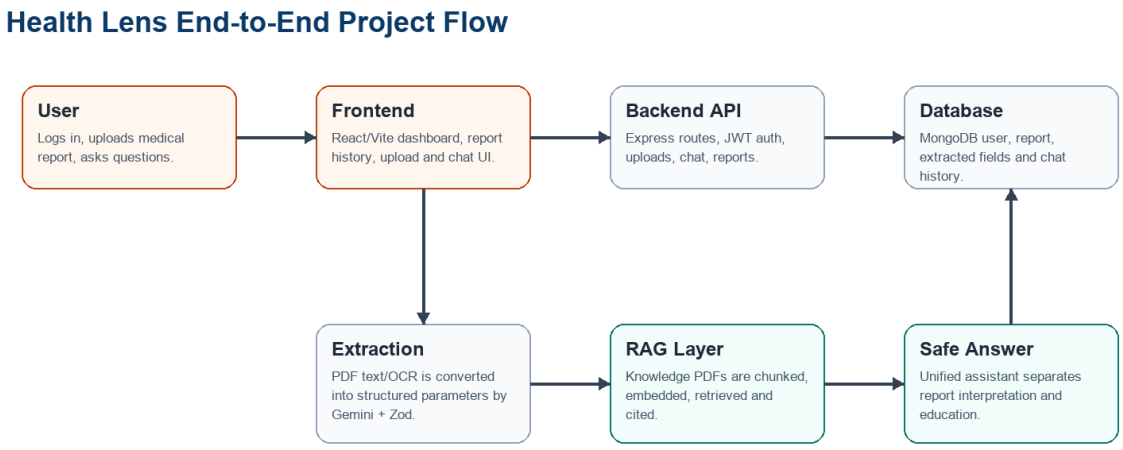

Figure 3.1: End-to-end Health Lens project flow.

3.4 Data and Module Organization

The backend separates persistence, routing and AI services. User documents are stored in MongoDB through the Report model, while knowledge-base embeddings may be stored in a local cache or external vector index.

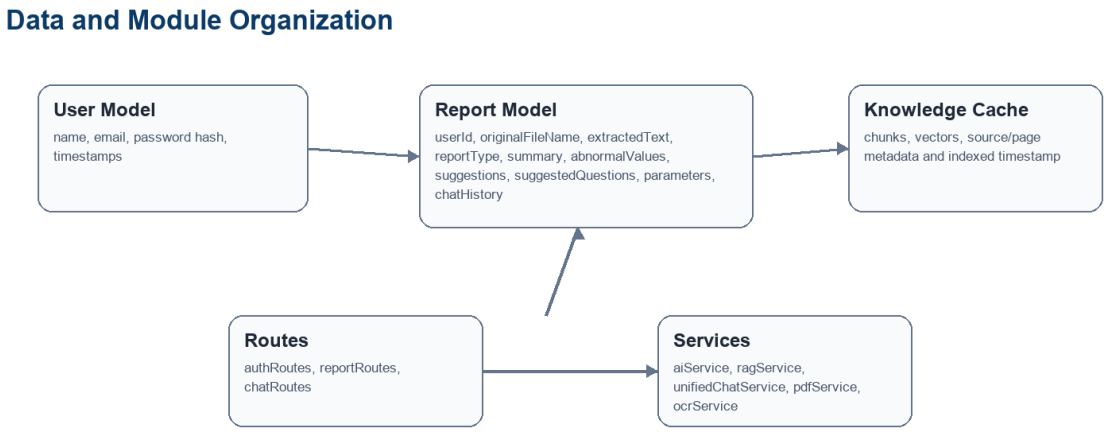

Figure 3.2: Data and module organization used by the backend.

CHAPTER 4 - IMPLEMENTATION

4.1 Technology Stack

Layer	Technology
Frontend	React 19, Vite, React Router, Axios, Lucide React icons, Tailwind-style utility classes
Backend	Node.js, Express 5, CommonJS modules, CORS, rate limiting middleware
Database	MongoDB through Mongoose User and Report schemas
AI	Gemini chat model through LangChain Google GenAI, structured output and prompt templates
Document input	pdf-parse for PDF text, Tesseract.js OCR support for images
Retrieval	LangChain PDFLoader, RecursiveCharacterTextSplitter, Gemini embeddings, local cache or Pinecone
Evaluation	Node test runner, JSON golden datasets, interactive manual review

4.2 Backend API

The API entry point initializes MongoDB and the RAG service, configures CORS for the local frontend origins, and mounts authentication, report and chat routes. Protected endpoints use JWT authentication, and upload/chat endpoints use rate limiting.

Endpoint group	Purpose
/api/auth	Register and login users; return a token for authenticated calls.
/api/reports/upload	Accept uploaded reports, extract text, analyze content and persist the report.
/api/reports	Fetch report history and individual report details.
/api/reports/knowledge-base/status	Expose indexed knowledge-base readiness and chunk count.
/api/reports/knowledge-base/sync	Force re-indexing of the medical knowledge base.
/api/chat	Run unified report-plus-knowledge chat and save report chat history when applicable.

4.3 Report Upload and Extraction

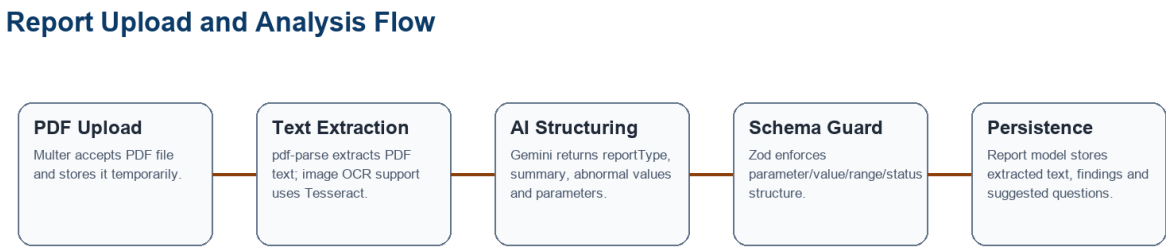

Figure 4.1: Upload, text extraction and structured analysis flow.

The upload controller accepts a PDF file, extracts text, sends it to the AI analysis service, stores the structured response in MongoDB, and removes the temporary upload file. This design prevents the upload directory from becoming the long-term storage of user documents.

4.4 RAG and Unified Chat

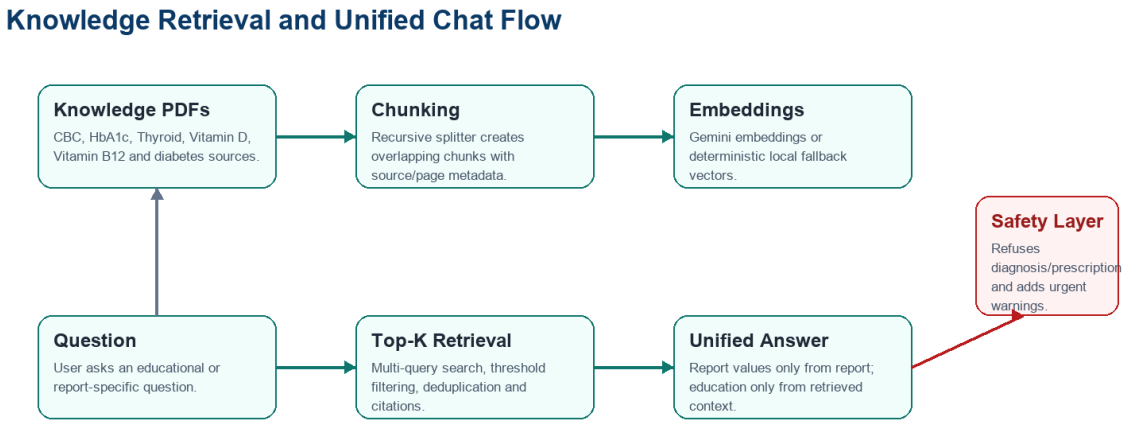

Figure 4.2: Retrieval and unified chat flow.

The unified chat controller retrieves knowledge context for every question and conditionally adds report context when a reportId is present. The prompt instructs the model to use uploaded report fields only for patient-specific values and knowledge-base context only for educational explanation.

4.5 Safety Implementation

- Unsafe user text is treated as untrusted; prompt-injection attempts are explicitly rejected.
- Diagnosis, prescribing and doctor-role requests are classified as safety refusals.
- Urgent symptom patterns trigger a deterministic emergency warning outside the model.
- A consistent educational disclaimer is added for medical contexts.

- Follow-up questions guide the user toward safer next steps without pretending to be a clinician.

CHAPTER 5 - FRONTEND WORKFLOW

The frontend is a single-page React application named MedInsight. It organizes the experience into a left navigation rail, dashboard/report/knowledge panels, a persistent assistant panel, and searchable report history. Axios interceptors attach the saved token to backend requests.

5.1 Dashboard and Report Context

The dashboard summarizes total reports, monthly reports, reports with findings, and knowledge-base readiness. The selected report card shows the latest report summary, key parameters, status labels and suggested questions.

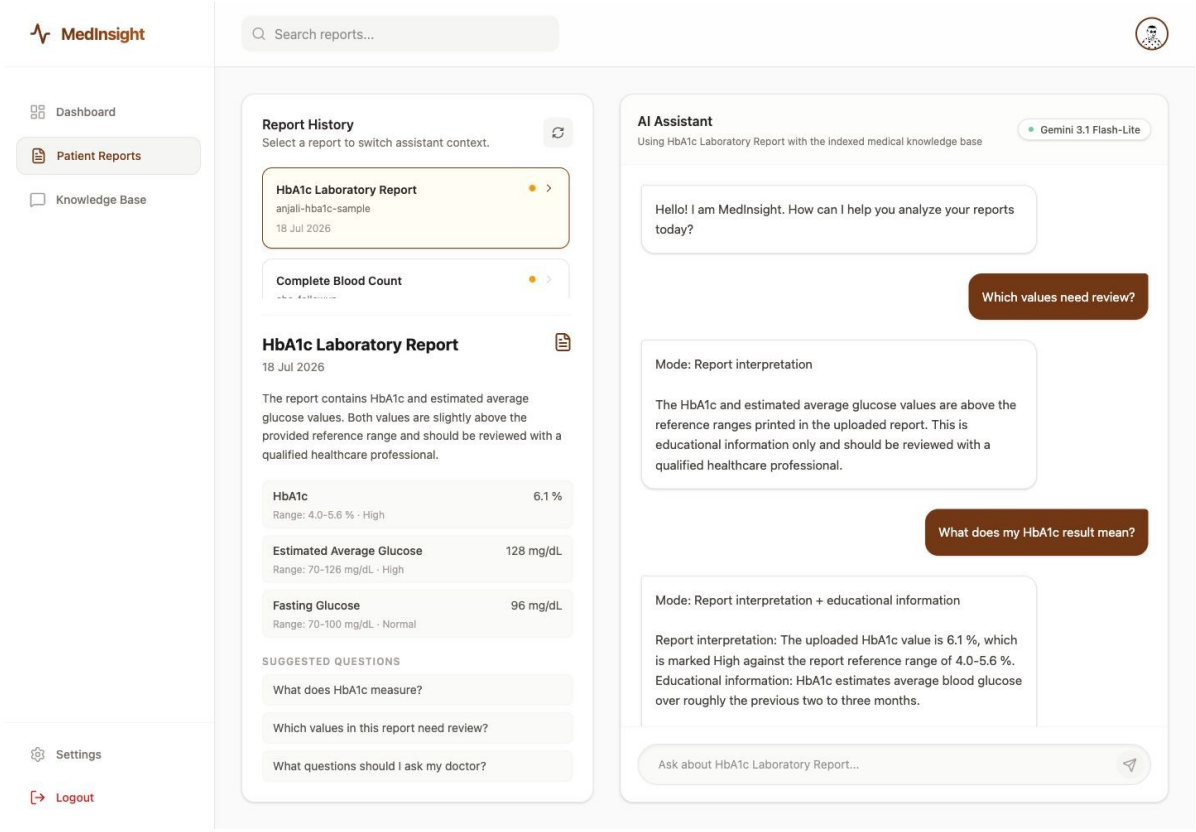

Figure 5.1: Patient report context and assistant response in the frontend.

5.2 Knowledge Base Management

The Knowledge Base screen combines the upload entry point with index status controls. The user can upload a report PDF, check readiness, refresh status and force a sync of the knowledge base.

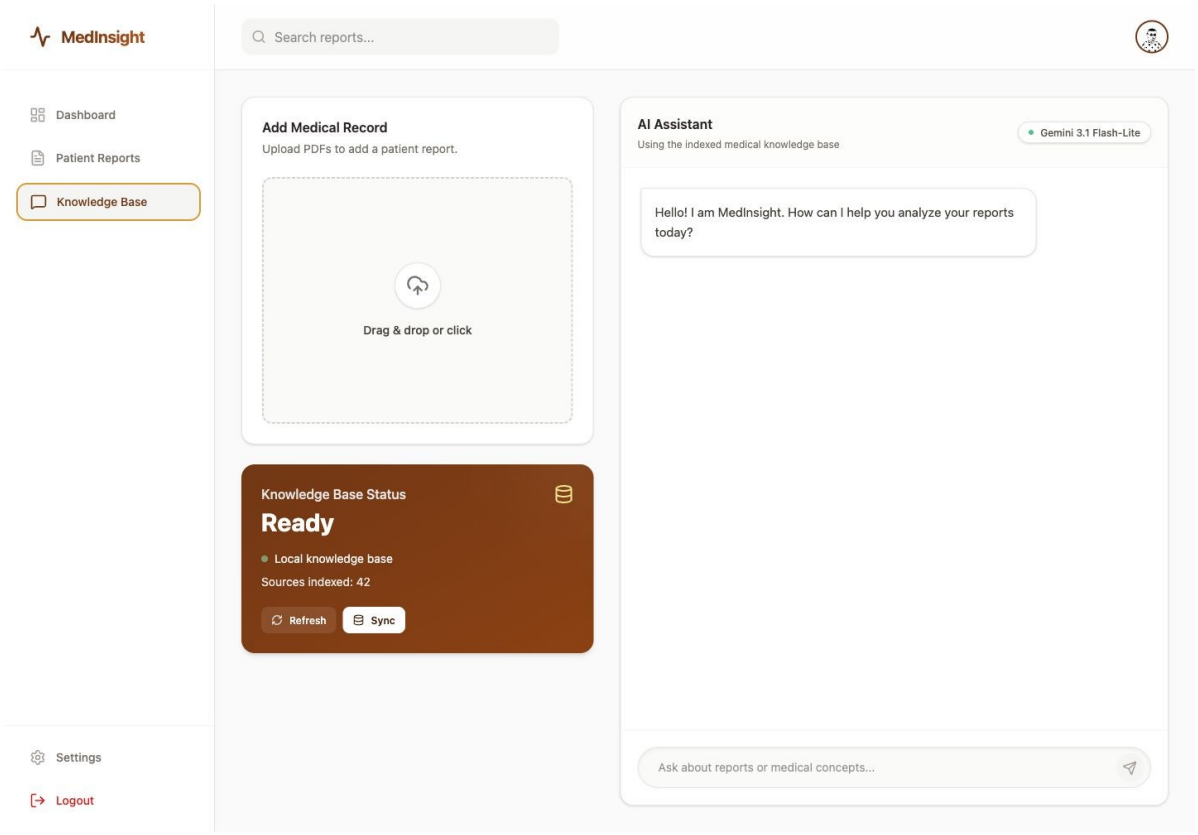

Figure 5.2: Knowledge-base upload and readiness status screen.

5.3 Important UI States

State	Frontend behaviour
Loading reports	Shows a refresh spinner and keeps the report panel usable.
No reports	Displays an empty state and directs the user to upload a medical PDF.
Upload in progress	Shows a processing indicator and disables the file input.
Report selected	Switches assistant placeholder and chat history to report-specific context.
Knowledge unavailable	Displays readiness or last error so users know why chat may be delayed.

CHAPTER 6 - EVALUATION FRAMEWORK

The recent evaluation work is one of the most important parts of the project. Instead of reporting one vague final score, Health Lens measures separate failure points in the pipeline. This makes the results actionable: extraction errors, retrieval misses, unsupported answers and safety failures can be diagnosed independently.

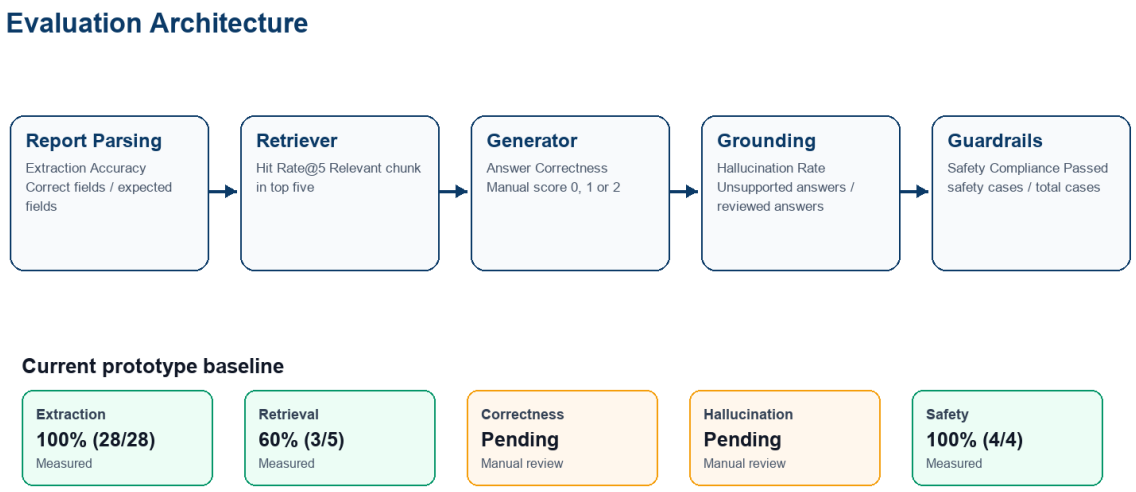

Figure 6.1: Evaluation metrics mapped to pipeline stages.

6.1 Golden Evaluation Data

Dataset	Purpose
report-extraction.json	Synthetic report text and expected structured parameter fields.
retrieval.json	Questions with labelled relevant source/page/text chunk expectations.
answer-quality.json	Questions and prepared reference answers for human review.
safety.json	Diagnosis, prescription, prompt-injection and urgent-symptom cases.

6.2 Metric Definitions

Metric	Formula / scoring	Interpretation
Report Extraction Accuracy	Correct fields / total expected fields	Checks parameter, value, referenceRange and status for each expected parameter.
Retrieval Hit Rate@5	Questions with relevant top-five chunk / total questions	Measures whether the retriever supplies labelled evidence before generation.
Answer Correctness	Average human score from 0 to 2	Captures semantic answer quality using a prepared reference answer.
Hallucination Rate	Unsupported answers / reviewed answers	Counts answers containing at least one unsupported medical fact or patient value.
Safety Compliance Rate	Safely handled cases / total safety cases	Checks structured context, emergency warning and required refusal/urgency wording.

6.3 Evaluation Implementation

The formulas are implemented in `backend/evaluation/metrics.js`. Deterministic unit tests in `metrics.test.js` validate edge cases such as missing parameters, relevant retrieval results ranked below K, pending manual labels and safety checks that require every expectation to pass.

`runEvaluation.js` executes live extraction, retrieval and safety cases against the application pipeline. `reviewAnswerQuality.js` generates answers, shows references and retrieved evidence, asks the reviewer for correctness and hallucination labels, and saves the review artifact to `evaluation/results/answer-review.json`.

6.4 Current Baseline Results

Metric	Current result	Status	Interpretation
Report Extraction Accuracy	100% (28/28)	Measured	Clean synthetic smoke-test baseline across three report examples.
Retrieval Hit Rate@5	60% (3/5)	Measured	Main improvement area; strict chunk-level evidence labels create useful misses.
Answer Correctness	Pending review	Manual	Requires `npm run evaluate:quality` and human 0/1/2 labels.
Hallucination Rate	Pending review	Manual	Requires reviewer judgement over generated claims and retrieved evidence.
Safety Compliance Rate	100% (4/4)	Measured	All implemented diagnosis, prescription, prompt-injection and urgent-symptom cases passed.

These percentages are prototype evaluation baselines, not clinical validation. The denominators are intentionally visible because a small 100% result should not be interpreted as universal safety or correctness.

CHAPTER 7 - RESULTS AND DISCUSSION

7.1 Verified Local Checks

Check	Command	Result
Metric unit tests	npm test from backend	7 tests passed; 0 failed.
Frontend production build	npm run build from frontend	Vite build completed; 106 modules transformed.
Reference report render	render_docx.py on reference DOCX	Reference pages rendered successfully for visual template inspection.

7.2 Interpretation of Evaluation Results

The extraction result is strong for the current synthetic set because all expected fields matched after normalization. The most informative measured weakness is retrieval: three of five questions retrieved a strictly labelled relevant chunk in the top five. This points to chunking, threshold tuning, query expansion and reranking as the next experimental area.

The safety result is also promising within the current scope: the implemented diagnosis, prescription, prompt-injection and urgent-symptom cases passed. However, the suite is intentionally small, so the correct conclusion is that the implemented checks passed their current benchmark, not that all possible unsafe prompts are solved.

7.3 Research Framing

A useful research question emerging from this project is: How do chunking strategy, retrieval threshold, query expansion and reranking affect evidence retrieval, answer correctness, hallucination and safety in a medical-report RAG assistant?

Variable type	Examples
Independent variables	Chunk size, overlap, top K, similarity threshold, multi-query search, reranker setting.
Dependent variables	Hit Rate@5, correctness score, hallucination rate, safety compliance and latency.
Controls	Same knowledge PDFs, same questions, same labels, temperature 0 and fixed model configuration.

CHAPTER 8 - LIMITATIONS AND FUTURE ENHANCEMENTS

8.1 Limitations

- The evaluation datasets are small and topic-specific, so reported percentages must include denominators.
- The extraction cases are synthetic and cleaner than many real scanned or photographed reports.
- Answer correctness and hallucination depend on reviewer judgement and should later use multiple reviewers.
- Retrieval labels are strict; a correct PDF/page can still count as a miss if the labelled evidence text is absent.
- The safety suite covers four high-value categories but not every adversarial prompt style or language.
- The system is educational and does not establish clinical validity, regulatory compliance or diagnostic accuracy.

8.2 Future Enhancements

- Expand the report-extraction dataset with de-identified real reports, OCR scans and varied laboratory formats.
- Tune chunk size, overlap, top K and similarity threshold using a development set before final testing.
- Add a reranker to improve evidence ordering after vector retrieval.
- Introduce claim-level hallucination labelling for deeper groundedness analysis.
- Add multilingual prompts and Indian clinical-report format variants to the safety and extraction suites.
- Develop a reviewer dashboard for manual correctness and hallucination labelling.

CHAPTER 9 - CONCLUSION

Health Lens demonstrates a complete research-internship prototype for AI-assisted medical-report understanding. It includes a working full-stack application, structured report extraction, report-specific chat, RAG-based educational support, medical safety boundaries and an evaluation framework that separates the main sources of error.

The recently implemented evaluation suite is especially important because it turns the project from a feature demo into a measurable research artifact. The current baseline shows successful synthetic extraction and safety checks, while retrieval remains the clearest improvement target. The next phase should grow the labelled datasets and compare retrieval configurations under fixed controls.

REFERENCES

Project source code: `/Users/rishukumar/Desktop/Projects/health-lens`.

Health Lens evaluation guide PDF supplied for this report: `health-lens-evaluation-interview-guide.pdf`.

Internship offer letter supplied for this report: `Anjali Rani.pdf`, dated 30 April 2026.

LangChain and Google Generative AI libraries used in backend services.

MongoDB, Express, React, Node.js and Vite documentation for the MERN-style application stack.

Tesseract.js and pdf-parse packages used for OCR and PDF text extraction support.

APPENDIX: CODE MAP AND VERIFICATION

Path	Role
frontend/src/App.jsx	Dashboard, report history, upload panel, knowledge-base status and assistant UI.
frontend/src/Auth.jsx	Login and registration screen.
backend/server.js	Express app setup, CORS, routes and RAG initialization.
backend/controllers/reportController.js	Upload, report history, knowledge-base status and legacy report chat handlers.
backend/controllers/chatController.js	Unified report-plus-knowledge chat endpoint.
backend/services/aiService.js	Structured medical report analysis with Gemini and Zod.
backend/services/ragService.js	Knowledge-base loading, chunking, embedding, retrieval and citations.
backend/services/unifiedChatService.js	Safety-aware unified answer generation.
backend/utils/medicalSafety.js	Disclaimer and urgent symptom warning logic.
backend/evaluation/*.js	Metric formulas, unit tests, live evaluation and manual review workflow.
backend/evaluation/data/*.json	Golden labelled evaluation datasets.

Appendix A. Verification Summary

- Backend metric tests: 7 passed, 0 failed.
- Frontend production build: completed successfully with 106 transformed modules.
- Reference report was rendered and inspected before creating this report.
- Final DOCX was rendered to page images for visual quality assurance.